

MANUAL TÉCNICO

El proyecto fué realizado usando java 21, para el backend, se utilizó angular par el frontend

y sql como base de datos

Se usó NetBeans como editor de código, Linux Pop Os 22 como sistema operativo

Base de datos

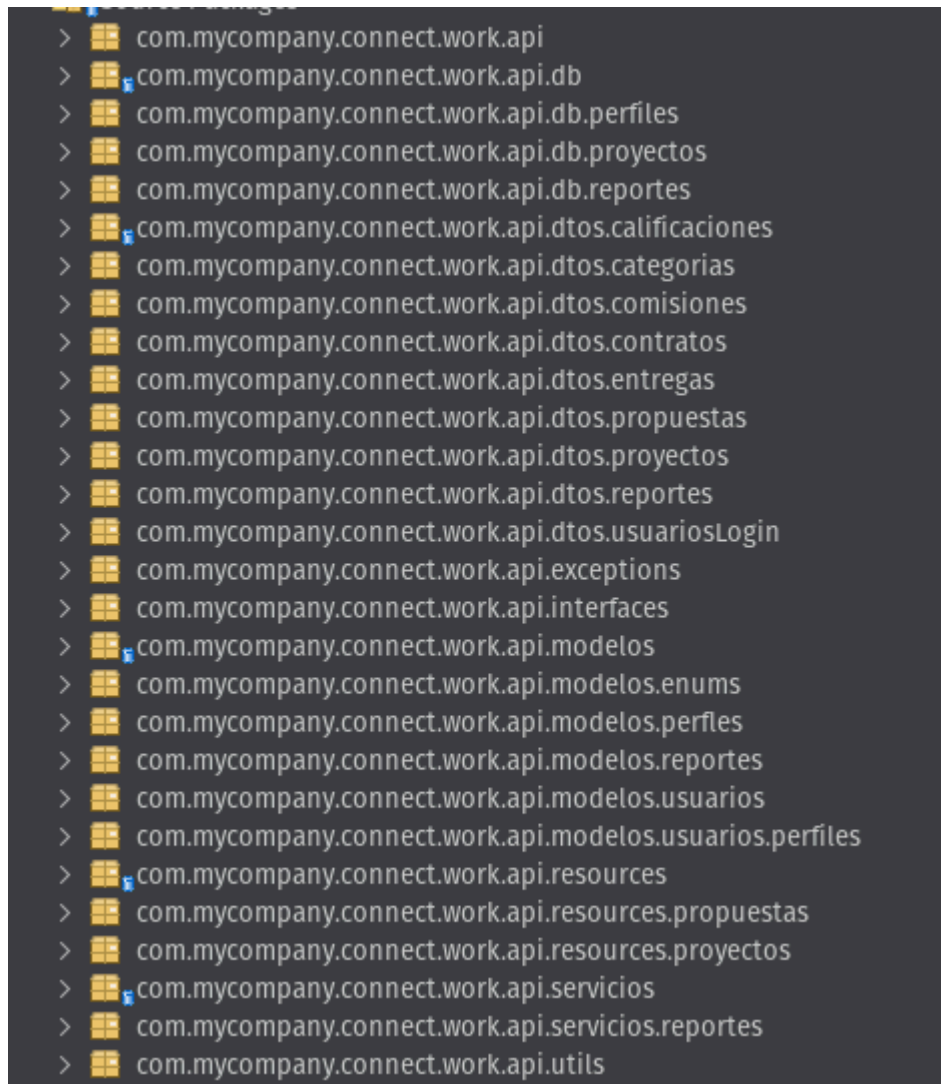
se crearon en total 30 tablas pero no se implementaron todas las cuales son:

```
+-----+
| Tables_in_connectWork |
+-----+
| archivo_entrega        |
| calificacion_freelancer |
| cambio_comision        |
| cancelacion_proyecto   |
| cartera_virtual        |
| categoria              |
| contrato               |
| entrega                |
| estado_entrega         |
| estado_propuesta       |
| estado_proyecto        |
| estado_solicitud       |
| habilidad              |
| habilidad_categoria    |
| habilidad_freelancer   |
| nivel_experiencia      |
| notificacion_freelancer |
| pago_proyecto          |
| perfil_cliente         |
| perfil_freelancer       |
| propuesta_proyecto     |
| proyecto               |
| recargo_tarjeta        |
| rechazo_entrega        |
| rol                    |
| solicitud_categoria    |
| solicitud_habilidad    |
| tabla_configuracion    |
| usuario_plataforma     |
| usuario_sistema        |
+-----+
```

Ordenamiento de paquetes código fuente

Backend

en el caso de los paquetes se dividen en paquetes de acceso de datos (DB), paquetes de service que son los encargados de llevar a cabo las validaciones y la lógica de negocio, además paquetes de dtos para traer y mandar datos al frontend, y por ultimo los resources que son los endpoints de la api.



service:

es una clase que recibe la entidad y realiza las validaciones necesarias antes de hacer registros en la db:

```
17
public class EntregasService extends CrudService {

    private ProyectosService proyectosService = new ProyectosService();
    private EntregasDB entregasDB = new EntregasDB();
    private RechazosEntregasDB rechazosDB = new RechazosEntregasDB();
    private ContratosService contratosService = new ContratosService();
    private PagosDB pagosDB = new PagosDB();
    private CarteraDigitalService carterasService = new CarteraDigitalService();

    public void crearEntrega(EntregaRequest entrega) throws DatosMuyLargosException, CamposVaciosException, ErrorDeLogicaException, DBException {

        this.revisarDatosCorrectos(entrega);

        int estadoProyecto = proyectosService.buscarEstadoProyecto(entrega.getIdProyecto());

        if(entregasDB.existeEntregaSinRevisar(entrega.getIdProyecto())){
            throw new ErrorDeLogicaException(" no puede realizar entregas, el cliente no ha revisado su ultima entrega");
        }

        if(estadoProyecto != EstadosProyecto.EN_PROGRESO.getId()){
            throw new ErrorDeLogicaException(" no puede realizar entregas, el proyecto no está en etapa de progreso");
        }

        int idEntrega = entregasDB.crear(entrega);

        ArrayList<ArchivoEntrega> archivos = entrega.crearArchivosEntregas(idEntrega);

        for (ArchivoEntrega archivo : archivos) {
            entregasDB.insertarArchivo(archivo.getIdEntrega(),archivo.getArchivo());
        }

        proyectosService.cambiarEstadoProyecto(entrega.getIdProyecto(), EstadosProyecto.ENTREGA_PENDIENTE.getId());
    }
}
```

en este ejemplo se puede ver la creación de una entrega.

Clase db:

esta es una clase que solo recibe la entidad, y realiza registros en la base de datos: no realiza validaciones a diferencia de una clase Crud service:

```
public class PagosDB implements CreacionEntidad<Pago> {

    private static final String CREAR_PAGO = "INSERT INTO pago_proyecto (fecha_pago, monto, comision, id_contrato) VALUES (?, ?, ?, ?)";

    @Override
    public void crear(Pago pago) throws DBException {
        try (Connection conn = ConexionDB.getConnection();
            PreparedStatement ps = conn.prepareStatement(CREAR_PAGO)) {

            ps.setDate(1, java.sql.Date.valueOf(pago.getFechaPago()));
            ps.setDouble(2, pago.getMonto());
            ps.setDouble(3, pago.getComision());
            ps.setInt(4, pago.getIdContrato());
            ps.executeUpdate();

        } catch (SQLException e) {
            throw new DBException("Error al registrar el pago: " + e.getMessage());
        }
    }
}
```

Clase DTO:

esta clase se para llevar datos del backend al frontend, a diferencia de la clase modelo, representan una clase mas completa y no con información mínima como son los modelos, en esta ocasión se usaron algunos modelos como dtos

```
public class CategoriaRequest {  
    public Categoria categoria;  
    public ArrayList<HabilidadCategoria> habilidades;  
  
    public CategoriaRequest(Categoria categoria, ArrayList<HabilidadCategoria> habilidades) {  
        this.categoria = categoria;  
        this.habilidades = habilidades;  
    }  
}
```

Clase MODELO:

representa la información mínima para llevar un registro en la base de datos

```
public class HabilidadCategoria extends Entidad {  
  
    private int idCategoria;  
    private int idHabilidad;  
  
    public int getIdCategoria() {  
        return idCategoria;  
    }  
  
    public void setIdCategoria(int idCategoria) {  
        this.idCategoria = idCategoria;  
    }  
  
    public int getIdHabilidad() {  
        return idHabilidad;  
    }  
  
    public void setIdHabilidad(int idHabilidad) {  
        this.idHabilidad = idHabilidad;  
    }  
  
    @Override  
    public boolean datosCompleto() {  
        return true;  
    }  
  
    @Override  
    public boolean datosTamañoCorrecto() {  
        return true;  
    }  
}
```

Clase RESOURCE:

representa un endpoint de la api, tiene metodos como get, put, delete, post

```
@WebServlet(name = "BusquedaProyectoResponseId", urlPatterns = {"/api/proyectoResponse/*"})
public class BusquedaProyectoResponseId extends HttpServlet {

    private BusquedasProyectosService service = new BusquedasProyectosService();
    private EscritorJson escritor = new EscritorJson();

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        String ruta = obtenerParametroRuta(req);
        if (!(ruta != null && esNumero(ruta))) return;

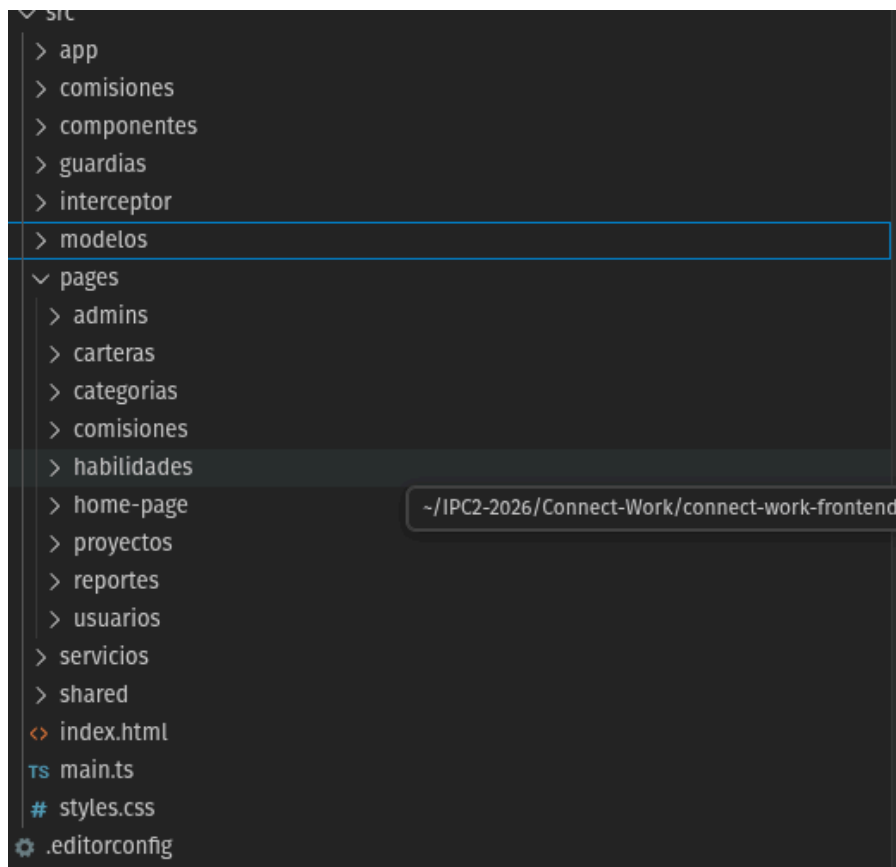
        try {
            Object objeto = service.buscarResponsePorId(Integer.valueOf(ruta));
            resp.setStatus(HttpServletResponse.SC_OK);
            escritor.escribirJsonConFecha(resp, objeto);
        } catch (DBException ex) {
            resp.setStatus(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);
            escritor.escribirError(ex.getMessage(), resp);
        }
    }

    private String obtenerParametroRuta(HttpServletRequest req) {
        String ruta = req.getPathInfo();

        if (ruta == null || ruta.equals("/")) {
            return null;
        } else {
            return ruta.substring(1);
        }
    }

    private boolean esNumero(String cadena) {
        try {
            Integer.valueOf(cadena);
            return true;
        } catch (NumberFormatException e) {
            return false;
        }
    }
}
```

Ordenamiento de paquetes FRONTEND



COMPONENTE:

los componentes se dividen en tarjetas y forms

las tarjetas muestran información importante, mientras los forms permiten la creación o edición de una entidad

PAGES:

una pagina contiene varios componentes y es el contenedor

existen varios tipos de pages

edit-page = pagina que busca una entidad por id y la pone en un componente si la encuentra para editar

form-page = es una pagina que el componente formulario de una entidad

page = una pagina normal que permite realizar varias acciones, ejemplo: pagina reservaciones, permite ver reservaciones, crear nueva reservación.